Solution to Singly Linked List - Push :

```
public class SinglyLinkedList {
 public Node head;
 public Node tail;
 public int size;

 public Node insertSinglyLinkedList(int
nodeValue) {
   head = new Node();
   Node node = new Node();
   node.next = null;
   node.value = nodeValue;
   head = node;
   tail = node;
   size = 1;
   return head;
 }

 // Insert Method SinglyLinkedList
 public void push(int nodeValue) {
   if (head == null) {
     insertSinglyLinkedList(nodeValue);
     return;
   } else {
     Node node = new Node();
     node.value = nodeValue;
     node.next = null;
     tail.next = node;
     tail = node;
     size++;
   }


 }
```

```
}
```

Solution to Singly Linked List - Pop :

```
public class SinglyLinkedList {
 public Node head;
 public Node tail;
 public int size;

 public Node insertSinglyLinkedList(int
nodeValue) {
   head = new Node();
   Node node = new Node();
   node.next = null;
   node.value = nodeValue;
   head = node;
   tail = node;
   size = 1;
   return head;
 }

 // Insert Method SinglyLinkedList
 public void push(int nodeValue) {
   if (head == null) {
     insertSinglyLinkedList(nodeValue);
     return;
   } else {
     Node node = new Node();
     node.value = nodeValue;
     node.next = null;
     tail.next = node;
```

```
      tail = node;
      size++;
    }


  }

  public Node pop() {
    if (head == null) {
      System.out.println("The SLL does not
exist");
      return null;
    }
    Node removeNode;
    Node currentNode;
    if (head == tail) {
      removeNode = head;
      head = tail = null;
    } else {
      currentNode = head;
      while (currentNode.next != tail) {
        currentNode = currentNode.next;
      }
      removeNode = currentNode.next;
      currentNode.next = null;
      tail = currentNode;
    }
    size--;

    return removeNode;

  }
```

```
}
```

Solution to Singly Linked List - Insert :

```
public class SinglyLinkedList {
 public Node head;
 public Node tail;
 public int size;

 public Node insertSinglyLinkedList(int
nodeValue) {
   head = new Node();
   Node node = new Node();
   node.next = null;
   node.value = nodeValue;
   head = node;
   tail = node;
   size = 1;
   return head;
 }

 // Insert Method SinglyLinkedList
 public void push(int nodeValue) {
   if (head == null) {
     insertSinglyLinkedList(nodeValue);
     return;
   } else {
     Node node = new Node();
     node.value = nodeValue;
     node.next = null;
     tail.next = node;
     tail = node;
     size++;
   }
```

```
 }

 public Node pop() {
   if (head == null) {
     System.out.println("The SLL does not
exist");
     return null;
   }
   Node removeNode;
   Node currentNode;
   if (head == tail) {
     removeNode = head;
     head = tail = null;
   } else {
     currentNode = head;
     while (currentNode.next != tail) {
       currentNode = currentNode.next;
     }
     removeNode = currentNode.next;
     currentNode.next = null;
     tail = currentNode;
   }
   size--;

   return removeNode;

 }


 //Get
 public Node get(int index) {
   if (index<0 || index >= size) {
     return null;
```

```
    }
    Node currentNode = head;
    for (int i=0; i<index; i++) {
      currentNode = currentNode.next;
    }
    return currentNode;
  }

  //Insert
  public boolean insert(int data, int index) {
    Node newNode = new Node();
    newNode.value = data;
    if (index<0 || index >= size) {
      return false;
    }
    if (head == null) {
      head = newNode;
      tail = newNode;
    } else {
      if (index == 0) {
        newNode.next = head;
        head = newNode;
      } else if (index == 1) {
        newNode.next = head.next;
        head.next = newNode;
      } else if (index == size) {
        tail.next = newNode;
        newNode.next = null;
        tail = newNode;
      } else {
        Node tempNode = head;
        int inx = 0;
        while (inx < index-1) {
          tempNode = tempNode.next;
          inx += 1;
```

```
            }
            Node nextNode = tempNode.next;
            tempNode.next = newNode;
            newNode.next = nextNode;
          }
        }
        size +=1;
        return true;
     }

    }
```

Solution to Singly Linked List - Get :

```
    public class SinglyLinkedList {
     public Node head;
     public Node tail;
     public int size;

     public Node insertSinglyLinkedList(int
    nodeValue) {
        head = new Node();
        Node node = new Node();
        node.next = null;
        node.value = nodeValue;
        head = node;
        tail = node;
        size = 1;
        return head;
     }

     // Insert Method SinglyLinkedList
     public void push(int nodeValue) {
        if (head == null) {
```

```
      insertSinglyLinkedList(nodeValue);
      return;
    } else {
      Node node = new Node();
      node.value = nodeValue;
      node.next = null;
      tail.next = node;
      tail = node;
      size++;
    }


  }

  public Node pop() {
    if (head == null) {
      System.out.println("The SLL does not
exist");
      return null;
    }
    Node removeNode;
    Node currentNode;
    if (head == tail) {
      removeNode = head;
      head = tail = null;
    } else {
      currentNode = head;
      while (currentNode.next != tail) {
        currentNode = currentNode.next;
      }
      removeNode = currentNode.next;
      currentNode.next = null;
      tail = currentNode;
    }
    size--;
```

```
    return removeNode;

  }


  //Get
  public Node get(int index) {
    if (index<0 || index >= size) {
      return null;
    }
    Node currentNode = head;
    for (int i=0; i<index; i++) {
      currentNode = currentNode.next;
    }
    return currentNode;
  }


}
```

Solution to Singly Linked List - Rotate :

```
public class SinglyLinkedList {
  public Node head;
  public Node tail;
  public int size;

  public Node insertSinglyLinkedList(int
nodeValue) {
    head = new Node();
    Node node = new Node();
    node.next = null;
```

```
    node.value = nodeValue;
    head = node;
    tail = node;
    size = 1;
    return head;
  }

  // Insert Method SinglyLinkedList
  public void push(int nodeValue) {
    if (head == null) {
      insertSinglyLinkedList(nodeValue);
      return;
    } else {
      Node node = new Node();
      node.value = nodeValue;
      node.next = null;
      tail.next = node;
      tail = node;
      size++;
    }


  }

  public Node pop() {
    if (head == null) {
      System.out.println("The SLL does not
exist");
      return null;
    }
    Node removeNode;
    Node currentNode;
    if (head == tail) {
      removeNode = head;
      head = tail = null;
```

```
    } else {
      currentNode = head;
      while (currentNode.next != tail) {
        currentNode = currentNode.next;
      }
      removeNode = currentNode.next;
      currentNode.next = null;
      tail = currentNode;
    }
    size--;

    return removeNode;

  }


  //Get
  public Node get(int index) {
    if (index<0 || index >= size) {
      return null;
    }
    Node currentNode = head;
    for (int i=0; i<index; i++) {
      currentNode = currentNode.next;
    }
    return currentNode;
  }

  //Insert
  public boolean insert(int data, int index) {
    Node newNode = new Node();
    newNode.value = data;
    if (index<0 || index >= size) {
      return false;
    }
```

```
  if (head == null) {
    head = newNode;
    tail = newNode;
  } else {
    if (index == 0) {
      newNode.next = head;
      head = newNode;
    } else if (index == 1) {
      newNode.next = head.next;
      head.next = newNode;
    } else if (index == size) {
      tail.next = newNode;
      newNode.next = null;
      tail = newNode;
    } else {
      Node tempNode = head;
      int inx = 0;
      while (inx < index-1) {
        tempNode = tempNode.next;
        inx += 1;
      }
      Node nextNode = tempNode.next;
      tempNode.next = newNode;
      newNode.next = nextNode;
    }
  }
  size +=1;
  return true;
}

//Rotate

public String rotate(int number) {
  int index = number;
  if (number < 0) {
```

```
      index = number + size;
    }
    if (index < 0 || index >= size) {
      return null;
    }
    if (index == 0) {
      return "No Rotation";
    }
    Node prevNode = head;
    for(int i=0; i<index-1; i++) {
      prevNode = prevNode.next;
    }
    if (prevNode == null) {
        return "No Rotation";
      }
    tail.next = head;
    head = prevNode.next;
    tail = prevNode;
    prevNode.next = null;
    return "Success";
  }


}
```

Solution to Singly Linked List - Set :

```
public class SinglyLinkedList {
 public Node head;
 public Node tail;
 public int size;

 public Node insertSinglyLinkedList(int
nodeValue) {
```

```
    head = new Node();
    Node node = new Node();
    node.next = null;
    node.value = nodeValue;
    head = node;
    tail = node;
    size = 1;
    return head;
  }

  // Insert Method SinglyLinkedList
  public void push(int nodeValue) {
    if (head == null) {
      insertSinglyLinkedList(nodeValue);
      return;
    } else {
      Node node = new Node();
      node.value = nodeValue;
      node.next = null;
      tail.next = node;
      tail = node;
      size++;
    }


  }

  public Node pop() {
    if (head == null) {
      System.out.println("The SLL does not
exist");
      return null;
    }
    Node removeNode;
    Node currentNode;
```

```
    if (head == tail) {
      removeNode = head;
      head = tail = null;
    } else {
      currentNode = head;
      while (currentNode.next != tail) {
        currentNode = currentNode.next;
      }
      removeNode = currentNode.next;
      currentNode.next = null;
      tail = currentNode;
    }
    size--;

    return removeNode;

  }


  //Get
  public Node get(int index) {
    if (index<0 || index >= size) {
      return null;
    }
    Node currentNode = head;
    for (int i=0; i<index; i++) {
      currentNode = currentNode.next;
    }
    return currentNode;
  }

  //Insert
  public boolean insert(int data, int index) {
    Node newNode = new Node();
    newNode.value = data;
```

```
    if (index<0 || index >= size) {
      return false;
    }
    if (head == null) {
      head = newNode;
      tail = newNode;
    } else {
      if (index == 0) {
        newNode.next = head;
        head = newNode;
      } else if (index == 1) {
        newNode.next = head.next;
        head.next = newNode;
      } else if (index == size) {
        tail.next = newNode;
        newNode.next = null;
        tail = newNode;
      } else {
        Node tempNode = head;
        int inx = 0;
        while (inx < index-1) {
          tempNode = tempNode.next;
          inx += 1;
        }
        Node nextNode = tempNode.next;
        tempNode.next = newNode;
        newNode.next = nextNode;
      }
    }
    size +=1;
    return true;
  }

  //Rotate
```

```
public String rotate(int number) {
  int index = number;
  if (number < 0) {
    index = number + size;
  }
  if (index < 0 || index >= size) {
    return null;
  }
  if (index == 0) {
    return "No Rotation";
  }
  Node prevNode = head;
  for(int i=0; i<index-1; i++) {
    prevNode = prevNode.next;
  }
  if (prevNode == null) {
      return "No Rotation";
    }
  tail.next = head;
  head = prevNode.next;
  tail = prevNode;
  prevNode.next = null;
  return "Success";
}

// Set

public boolean set(int index, int value) {
  if (head == null) {
    head.value = value;
    tail.value = value;
  } else {
    Node currentNode = head;
    for (int i =0; i<index; i++) {
      currentNode = currentNode.next;
```

```
        if (currentNode == null) {
          return false;
        }
      }
      currentNode.value = value;
    }
    return true;
  }



}
```

Solution to Singly Linked List - Remove :

```
public class SinglyLinkedList {
 public Node head;
 public Node tail;
 public int size;

 public Node insertSinglyLinkedList(int
nodeValue) {
   head = new Node();
   Node node = new Node();
   node.next = null;
   node.value = nodeValue;
   head = node;
   tail = node;
   size = 1;
   return head;
```

```
}

public void push(int nodeValue) {
  if (head == null) {
    insertSinglyLinkedList(nodeValue);
    return;
  } else {
    Node node = new Node();
    node.value = nodeValue;
    node.next = null;
    tail.next = node;
    tail = node;
    size++;
  }


}



//Remove
public Node remove(int index) {
  if (index<0 || index >= size) {
    return null;
  }
  Node removeNode;
  Node currentNode;
  size--;
  if (index == 0) {
    removeNode = head;
    if (head == tail) {
      head = tail = null;
    } else {
      head = head.next;
    }
```

```
      return removeNode;
    } else {
      currentNode = head;
      int indx = 0;
      while (indx < index - 1) {
        currentNode = currentNode.next;
        indx ++;
      }
      removeNode = currentNode.next;
      Node nextNode = currentNode.next;
      currentNode.next = nextNode.next;
      return removeNode;


    }
  }


}
```